

Dual Conditioning for MusicGen: Combining Raw-Text Semantics with Mood-Embedding Steering

A New Approach to Stage C, Building on the Bridge Adapter

EmoMV Project

July 6, 2026

Contents

Executive Summary	1
1 Motivation	2
1.1 What the current design gives up	2
1.2 Why MusicGen is a natural fit for adding this back	2
2 Proposed Mechanism	2
2.1 Verified against the installed <code>transformers</code> source	2
2.2 The combined forward pass	3
2.3 What stays frozen, what gets retrained	3
2.4 Why the bridge must be retrained, not reused as-is	4
3 Training Data for the Combined Bridge	4
3.1 Reuse the existing paired data	4
3.2 Two conditioning regimes worth training/evaluating separately	4
4 Evaluation Protocol	4
5 Feasibility Summary	5
6 Recommended Next Steps	5
Open Assumptions to Confirm	6
Recap: Referenced Prior Design	6

Executive Summary

The current Stage C design (Milestone 1, implemented in `project/models/bridge.py` and `08_bridge.ipynb`) discards the original input text entirely once it has been classified. A video or sentence is mapped, via the frozen Stage A/B encoders and adapters, to a single 256-d point z in the shared mood space; the bridge network turns z into a synthetic

(T , 768) pseudo-sequence that *replaces* MusicGen’s normal T5 text-conditioning input outright (`encoder_outputs=(bridge_output,)`, bypassing the T5 call). This is sufficient signal for *which of 5 mood classes* to generate toward, but it throws away everything else in the input text — instrumentation, tempo, genre, and any descriptive detail beyond the mood label.

Proposed change: keep the bridge exactly as designed for mood steering, but stop bypassing MusicGen’s own frozen T5 encoder. Run the raw input text through T5 *as MusicGen already knows how to consume it*, run the same text (or a video’s aligned embedding) through the existing bridge for mood steering, and **concatenate the two conditioning sequences** along the time axis before handing them to the frozen decoder’s cross-attention. This is not a redesign of Stage C — it is a one-line change in what gets passed as `encoder_outputs`, verified directly against the installed `transformers` MusicGen source (Section 2).

Verdict: feasible with no MusicGen fine-tuning of any kind. MusicGen’s T5 encoder and LM decoder remain fully frozen, exactly as in the existing plan. The only thing that must be *retrained* is the small bridge network itself (already a trained component, not a frozen one) — because it has so far only ever been trained as the *sole* conditioning signal, and needs to learn to steer mood *alongside* real T5 tokens rather than in place of them.

Component	Status under this proposal	Notes
T5 text encoder (MusicGen’s own)	Frozen, now actually used	Currently unused at inference time; the bridge fully replaces it
EnCodec tokenizer / LM decoder	Frozen (unchanged)	Same as current plan
Bridge network	Retrain (small, cheap)	Same module, new training context (concatenated conditioning)
Stage B alignment adapters	Frozen (unchanged)	Still the source of the mood vector z

1 Motivation

1.1 What the current design gives up

Section 6 of the existing cross-modal alignment plan is explicit about what label-anchored contrastive alignment does *not* buy: two very different scenes sharing a label (“a rainy evening with tea” vs. “a quiet mountain hike,” both *relax*) land at nearly the same point in the shared space. That is fine for the alignment space itself, whose job is coarse mood separation. But the current Stage C bridge takes *only* that collapsed point as MusicGen’s entire conditioning input. Any richer information present in the original text — “a slow acoustic guitar piece for a quiet mountain hike” carries genre and instrumentation cues that “relax” alone does not — is computed by T5 internally in stock MusicGen usage, but never reaches the decoder in this project’s current pipeline, because `encoder_outputs` is overwritten by the bridge before the forward pass ever gets there.

1.2 Why MusicGen is a natural fit for adding this back

MusicGen does not consume a single vector for text conditioning; it consumes a *sequence* of T5 hidden states, cross-attended by every decoder layer, with an accompanying padding mask that already handles variable-length conditioning inputs. Concatenating a second sequence (the bridge’s

mood tokens) onto the first (T5’s semantic tokens) is not a structural change to how MusicGen attends over conditioning — it is the same mechanism MusicGen already uses for batched, variable-length text prompts, just with two sources instead of one.

2 Proposed Mechanism

2.1 Verified against the installed transformers source

Inspecting `MusicgenForConditionalGeneration.forward` directly (transformers/models/musicgen/modeling_musicgen.py) confirms the exact contract:

- If `encoder_outputs` is not supplied, MusicGen calls `self.text_encoder` (T5) itself to produce it from `input_ids/attention_mask`.
- If `encoder_outputs` *is* supplied (a tuple), it is wrapped directly into a `BaseModelOutput` and used as-is — this is the hook the current bridge already exploits.
- The top-level `attention_mask` is used twice: to zero out masked positions of `encoder_hidden_states` (`encoder_hidden_states *= attention_mask[..., None]`), and passed straight through to the decoder as `encoder_attention_mask` for cross-attention.

Nothing in this contract assumes `encoder_outputs` came from a single source or has a particular sequence length. A concatenation of two sequences with a concatenated mask is a first-class supported input, not a workaround.

2.2 The combined forward pass

For an input consisting of raw text (and/or, symmetrically, a video’s aligned embedding):

1. **Semantic branch (new):** tokenize the raw text and run it through MusicGen’s own frozen `text_encoder` (T5) to get $(B, T_{\text{text}}, 768)$ hidden states plus its padding mask. This is exactly what stock MusicGen does with a text prompt — nothing new is trained here.
2. **Mood branch (existing, unchanged):** encode the same input through the frozen Stage A/B pipeline to the shared 256-d vector z (video adapter or text adapter, whichever modality applies), then through the existing bridge to get $(B, T_{\text{bridge}}, 768)$, with an all-ones mask (as today).

3. Combine:

$$\text{cond} = [\text{T5}(\text{text}); \text{bridge}(z)] \in \mathbb{R}^{B \times (T_{\text{text}} + T_{\text{bridge}}) \times 768}, \quad \text{mask} = [\text{mask}_{\text{T5}}; \mathbf{1}_{T_{\text{bridge}}}]$$

4. Pass `encoder_outputs=(cond,)` and `attention_mask=mask` to `MusicgenForConditionalGeneration.forward` (training) or `.generate` (inference), exactly where the bridge output alone is passed today.

2.3 What stays frozen, what gets retrained

Component				Trainable?
MusicGen T5 text encoder				Frozen (unchanged)
MusicGen LM decoder (cross-attention, self-attention, everything)				Frozen (unchanged)
EnCodec tokenizer				Frozen (unchanged)
Stage	B	video/text	adapters	Frozen (unchanged)
(AlignmentModel)				
Bridge		network		Retrained (same module, new training context)
(EmbeddingToConditioningBridge)				

No MusicGen parameter ever receives a gradient under this proposal, matching the constraint of avoiding full or partial fine-tuning of the generative model. The retraining burden is identical in kind to what Milestone 1 already requires — only the bridge’s training loop changes, because the conditioning sequence it is optimized against now includes real T5 tokens alongside it rather than standing alone.

2.4 Why the bridge must be retrained, not reused as-is

The current checkpoint (`bridge_best.pt`) was trained with the bridge’s output as the *entire* cross-attention context — the decoder’s teacher-forced loss shaped the bridge to carry all conditioning information by itself. Under the combined scheme, the decoder will have real semantic tokens present in the same context. Reusing the old checkpoint unchanged risks two failure modes:

- The bridge continues trying to encode content the T5 tokens already provide, wasting its limited capacity redundantly.
- The two token groups are not adapted to coexist in the same attention context, so the decoder may attend inconsistently between them (e.g. ignoring one entirely, since nothing in the old training objective taught it to blend both).

Retraining is a rerun of `train_bridge.py/08_bridge.ipynb`’s existing loop with one change to the forward call (Section 2) — no new data collection, no new architecture, no new frozen-vs-trainable boundary to design.

3 Training Data for the Combined Bridge

3.1 Reuse the existing paired data

No new pairing problem is introduced. The same MATCH-clip (mood label, music audio) pairs used for the current bridge training (Section 6) remain the training target. The only addition is that each training example must now also carry its raw text — for video-derived examples this can be a caption, a template string built from metadata, or (if unavailable) the class-name text used for the existing “TEXT anchors” evaluation; for text-mood-dataset examples, the raw sentence is already the natural choice for the semantic branch.

3.2 Two conditioning regimes worth training/evaluating separately

1. **Video-anchored:** raw text = a caption or template describing the clip (if available) or omitted (semantic branch degenerates to an empty/short sequence); mood vector $z = \text{encode_video}$. This isolates whether the bridge, trained with a semantic branch present, still steers mood correctly when the semantic branch carries little information.
2. **Text-anchored:** raw text = the actual input sentence; mood vector $z = \text{encode_text}$ of the same sentence. This is the regime the new approach is really aimed at — letting T5 read the sentence’s real content while the mood branch supplies an explicit emotion signal independent of whether the sentence names the emotion outright.

Because Stage B’s shared space is meant to be modality-agnostic, both regimes route through the same trained bridge; the difference is only which adapter produced z and what text is fed to T5.

4 Evaluation Protocol

Do not evaluate solely on the existing teacher-forced LM loss; that loss cannot distinguish “the decoder generates coherent audio” from “the decoder generates audio that reflects the requested mood.” Reuse and extend the current qualitative protocol from `08_bridge.ipynb`:

1. **Ablation A — mood branch removed.** Generate with only the T5 sequence (`cond = T5(text)`), i.e. stock MusicGen behavior). This is the baseline the combined approach must beat on mood-consistency, since it is what MusicGen already does for free with no training at all.
2. **Ablation B — semantic branch removed.** Generate with only the bridge sequence (`cond = bridge(z)`), i.e. the current Milestone 1 design. This is the baseline the combined approach must not regress against.
3. **Combined.** Generate with both concatenated. Listen for: (a) does content specificity improve over Ablation B (e.g. an explicit instrument or genre word in the text actually surfaces audibly), and (b) does mood-consistency hold up to Ablation A, especially for text that does not name the emotion directly.
4. **Attention diagnostic (optional but cheap).** Inspect `cross attentions` returned by `Seq2SeqLMOutput` to check that decoder layers are not attending to one branch exclusively — a collapse to one branch is the clearest sign the bridge needs more retraining or a rebalanced loss.

5 Feasibility Summary

Component	Verdict	Key resource you have	Key open risk
Combined conditioning (concat T5 + bridge)	Feasible	Verified native support in MusicgenForConditionalGeneration; no new frozen/trainable boundary	None architecturally; correctness depends on generation forward not
Bridge retraining under new objective	Feasible	Same training loop, same paired MATCH-clip data, same compute footprint as current Milestone 1	Old checkpoint is not reusable as-is (Section 2.4); must retrain from scratch or from a warm start
Raw text for video-anchored examples	Feasible, higher risk	Text-mood dataset already supplies real sentences for the text-anchored regime	EmoMV videos may lack per-clip captions; may need templated or class-name placeholder text

6 Recommended Next Steps

1. Add a small helper that runs `musicgen.text_encoder` directly on tokenized raw text (mirroring what `forward` does internally when `encoder_outputs` is `None`), returning hidden states *and* the padding mask.
2. Change `run_epoch` in `train_bridge.py/08_bridge.ipynb` to build `cond/attn_mask` as the concatenation in Section 2 instead of `bridge(shared_z)` alone, and retrain.
3. Run the three-way ablation in Section 4 (T5-only, bridge-only, combined) on a handful of held-out examples before deciding this replaces Milestone 1’s single-source design.
4. Only if the combined approach is adopted: update `project/evaluation/generate_samples.py` and the notebook’s generation cells to build the same concatenated conditioning at inference time.

Open Assumptions to Confirm

- This plan assumes MusicGen’s T5 text encoder is reachable directly as `musicgen.text_encoder` in the installed `transformers` version (confirmed by source inspection at time of writing) — worth a one-line sanity check against whatever version is pinned in `requirements.txt` before implementing.
- This plan assumes per-clip captions are not currently available for EmoMV video; if they are (or can cheaply be generated, e.g. via an off-the-shelf video-captioning model), the video-anchored regime in Section 3.2 becomes strictly more informative and should use real captions rather than template/class-name placeholders.

- This plan assumes the existing Stage B retrieval gate (R@1 well above chance, per the cross-modal alignment plan’s own evaluation protocol) has been or will be confirmed before relying on `encode_text`-derived z as a trustworthy mood signal in the text-anchored regime.

Recap: Referenced Prior Design

Stage B pairing recap

For readers who have not just reviewed `cross_modal_alignment_plan.pdf`: Stage B trains two small adapters (video 512-d $\rightarrow D=256$, text 768-d $\rightarrow D=256$) with a label-anchored supervised contrastive loss, since no instance-level (video, caption) pairs exist — only shared 5-class labels. This gives coarse cross-modal mood alignment, not fine-grained semantic alignment within a class.

Stage C Milestone 1 recap

Stage C’s first milestone trains a bridge network $\mathbb{R}^{256} \rightarrow \mathbb{R}^{T \times 768}$ that maps a shared-space vector into MusicGen’s T5-conditioning slot, using EmoMV’s MATCH-clip audio (video emotion = music emotion by construction) as the paired training target, with MusicGen itself fully frozen throughout. This document proposes extending that milestone’s conditioning input, not replacing its data source or its frozen/trainable boundary.